BENEMÉRITA UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
Departamento de Ciencias Computacionales

Actividad de Aprendizaje 12. Programa 6. Productor-Consumidor

2026-B
Sección D09
Sistemas Operativos
Código de la materia: IL366

P R E S E N T A (N)
ICOM - Velázquez Vazquéz Valeria - 224785718

PROFESOR(A)
Becerra Velázquez Violeta del Rocío

Guadalajara, Jalisco, Mx.
29 de Octubre de 2026

Índice

Objetivo de la actividad..... 3
Notas acerca del lenguaje.....3
Arquitectura y estructuras de datos.....3
Reto principal y cómo se solucionó.....4
Capturas de pantalla.....5
Conclusiones.....7
Bibliografía..... 7

Tabla de Imágenes

Ilustración 1. El productor trabaja dentro del contenedor.....5
Ilustración 2. El consumidor trabaja dentro del contenedor.....6
Ilustración 3. Intento fallido: el consumidor no puede entrar.....6
Ilustración 4. Buffer circular: el productor regresa a la casilla 1.....7

Objetivo de la Actividad

Simular el problema clásico de sincronización Productor-Consumidor. Un único productor y un único consumidor comparten un contenedor con capacidad para 18 elementos que funciona como un buffer circular y acotado: al llegar a la casilla 18, ambos continúan en la casilla 1. El productor coloca elementos (letras, números y caracteres especiales) y el consumidor los retira, siempre en orden y continuando en la casilla donde se quedó cada uno.

Las reglas que se deben respetar son las de cualquier recurso compartido: solo uno puede estar dentro del contenedor a la vez (exclusión mutua), el productor solo puede entrar si hay espacio y el consumidor solo si hay producto. Ambos duermen tiempos aleatorios; al despertar intentan entrar y, si pueden, producen o consumen de 3 a 6 elementos. En pantalla se muestran el contenedor con sus casillas numeradas, el estado de cada uno (dormido, intentando entrar o trabajando) y mensajes de todo lo que ocurre. El programa termina al presionar la tecla ESC.

Notas acerca del Lenguaje

El programa se desarrolló en Python con PySide6 (Qt). El paso del tiempo se controla con un QTimer que dispara una unidad de tiempo cada 600 milisegundos. En cada unidad, el productor y el consumidor avanzan un paso: descuentan su tiempo de sueño, intentan entrar o procesan un elemento. Se optó por esta simulación por unidades de tiempo en lugar de hilos reales porque Qt solo permite modificar la interfaz desde el hilo principal y porque así el comportamiento es reproducible: con la misma semilla aleatoria se obtiene exactamente la misma secuencia, lo que permitió probar el modelo de forma automática.

El contenedor se dibuja con una clase propia que hereda de QWidget y sobrescribe paintEvent; con QPainter se trazan las 18 casillas numeradas, el elemento de cada una y dos marcas: P indica la siguiente casilla donde producirá el productor y C la siguiente de donde consumirá el consumidor. La casilla en la que se acaba de trabajar se resalta con el color del agente. La bitácora de mensajes es un QListWidget, y la tecla ESC se detecta sobrescribiendo keyPressEvent en la ventana principal.

Arquitectura y Estructuras de Datos

Se mantuvo la organización en modelo, vista y controlador de las actividades anteriores:

Archivo	Responsabilidad
config.py	Capacidad (18), rango de elementos por entrada (3 a 6), rango de sueño y productos posibles.
models/buffer.py	CircularBuffer: lista de 18 casillas con índices de entrada y salida que avanzan en módulo 18.
models/simulation.py	ProducerConsumer: estados de cada agente, exclusión mutua y bitácora de mensajes.
views/container_widget.py	Dibujo del contenedor numerado con las marcas P y C.
views/main_window.py	Paneles del productor y del consumidor, mensajes y tecla ESC.
controllers/sim_controller.py	QTimer que avanza la simulación y cierre del programa con ESC.
tests/test_simulation.py	Pruebas automáticas del modelo.

Tabla 1. Estructura del proyecto

El contenedor (CircularBuffer) guarda una lista de 18 casillas, el número de casillas ocupadas y dos índices: entrada, que es la siguiente casilla del productor, y salida, que es la siguiente casilla del consumidor. Al poner o quitar un elemento, el índice correspondiente avanza con la operación (i + 1) mod 18, lo que produce el comportamiento circular. Cada agente es un objeto con su estado, el tiempo de sueño restante, los elementos que le faltan en la entrada actual y el total procesado. La exclusión mutua se representa con el atributo dentro de la simulación, que indica quién ocupa el contenedor y funciona como un semáforo binario (mutex); el número de casillas ocupadas y libres cumple el papel de los semáforos lleno y vacío de la solución clásica.

Reto Principal y Cómo se Solucionó

El reto principal fue modelar correctamente los estados de cada agente y las condiciones para entrar. Cada uno pasa por un ciclo de tres estados: dormido, intentando entrar y trabajando. Al terminar su tiempo de sueño pasa a «intentando entrar» y, en la siguiente unidad de tiempo, se verifican sus condiciones: que el contenedor esté libre y que haya espacio (productor) o producto (consumidor). Si no puede entrar, se registra el motivo en los mensajes y vuelve a dormir un tiempo aleatorio; si puede, toma el contenedor y elige al azar cuántos elementos procesará (de 3 a 6).

Un segundo punto fue decidir qué pasa si, ya dentro, el contenedor se llena o se vacía antes de completar sus elementos. En ese caso el agente procesa solo los que puede, sale del contenedor dejando constancia del motivo y vuelve a dormir; así nunca se produce en un contenedor lleno ni se consume de uno vacío. Además, el orden en que se atiende a los dos agentes en cada unidad de tiempo se alterna, para que ninguno tenga ventaja cuando despiertan al mismo tiempo.

Para verificar el programa se escribieron pruebas automáticas que, con 30 semillas aleatorias y miles de unidades de tiempo, comprueban que nunca trabajan los dos a la vez, que las casillas ocupadas siempre coinciden con producidos menos consumidos, que los elementos se consumen exactamente en el mismo orden en que se produjeron, que cada entrada procesa de 3 a 6 elementos y que el contenedor da la vuelta de la casilla 18 a la 1. Al ejecutar el programa en

Windows para tomar las capturas se corrigieron además dos detalles: un error de sintaxis en la hoja de estilo de los paneles y el resaltado de la casilla de una entrada anterior cuando un agente volvía a entrar.

Capturas de Pantalla

Las siguientes ilustraciones corresponden a la ejecución del programa en Windows.

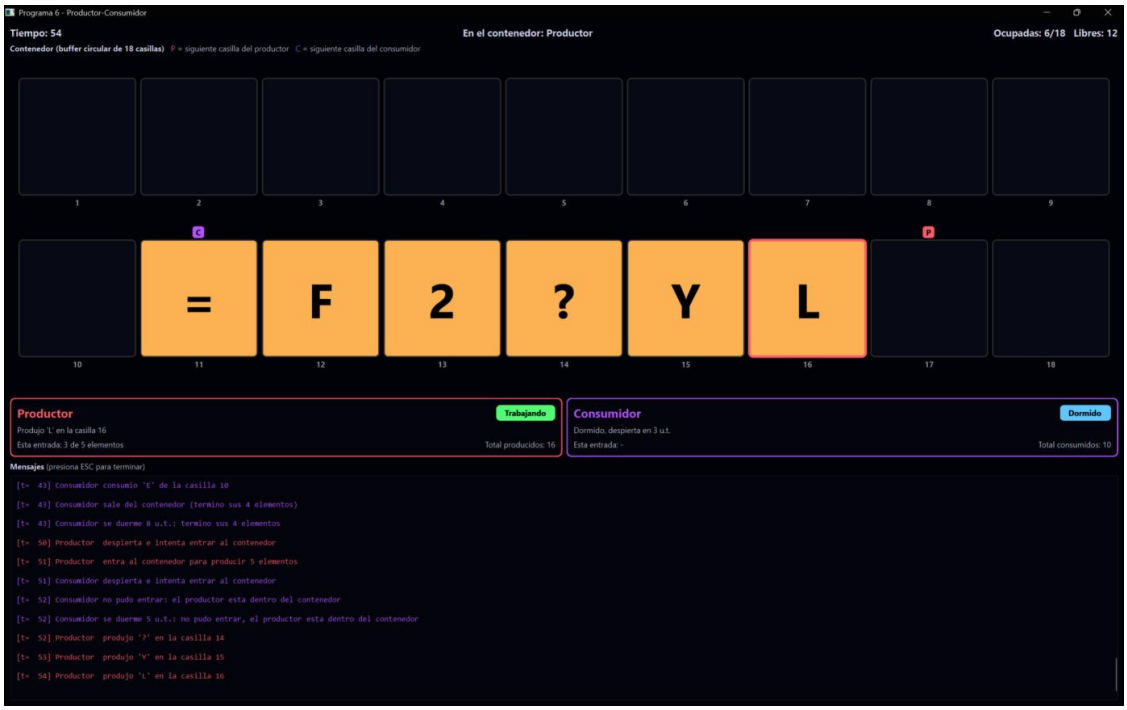


Ilustración 1. El productor trabaja dentro del contenedor

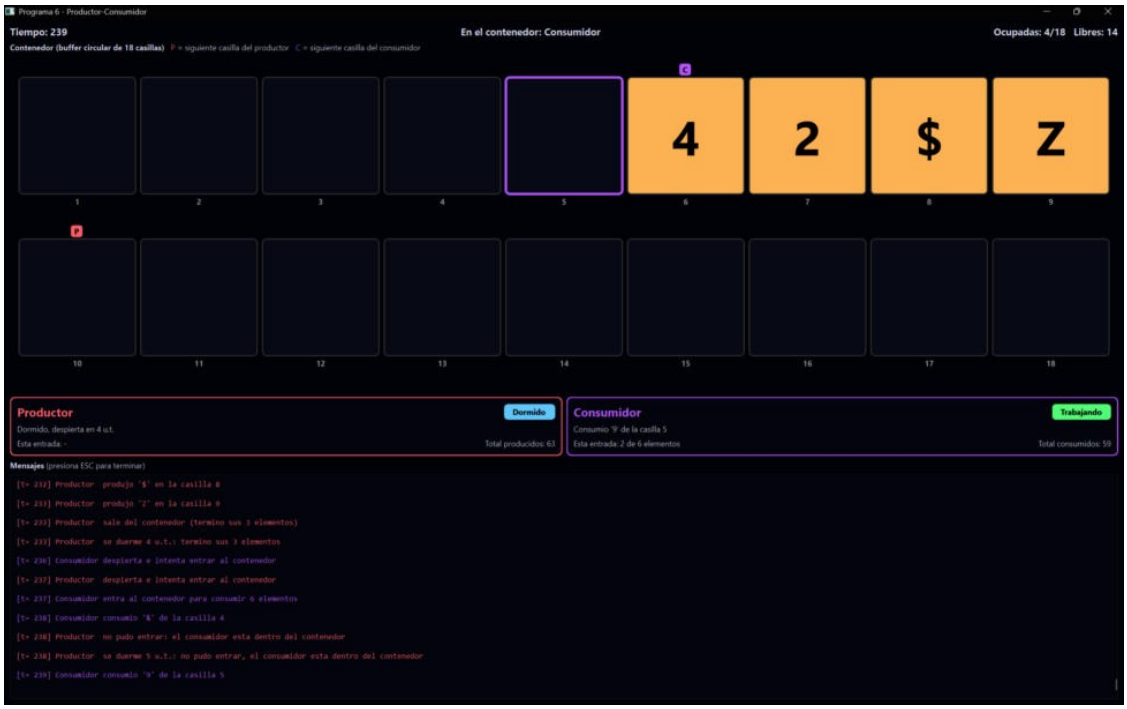


Ilustración 2. El consumidor trabaja dentro del contenedor

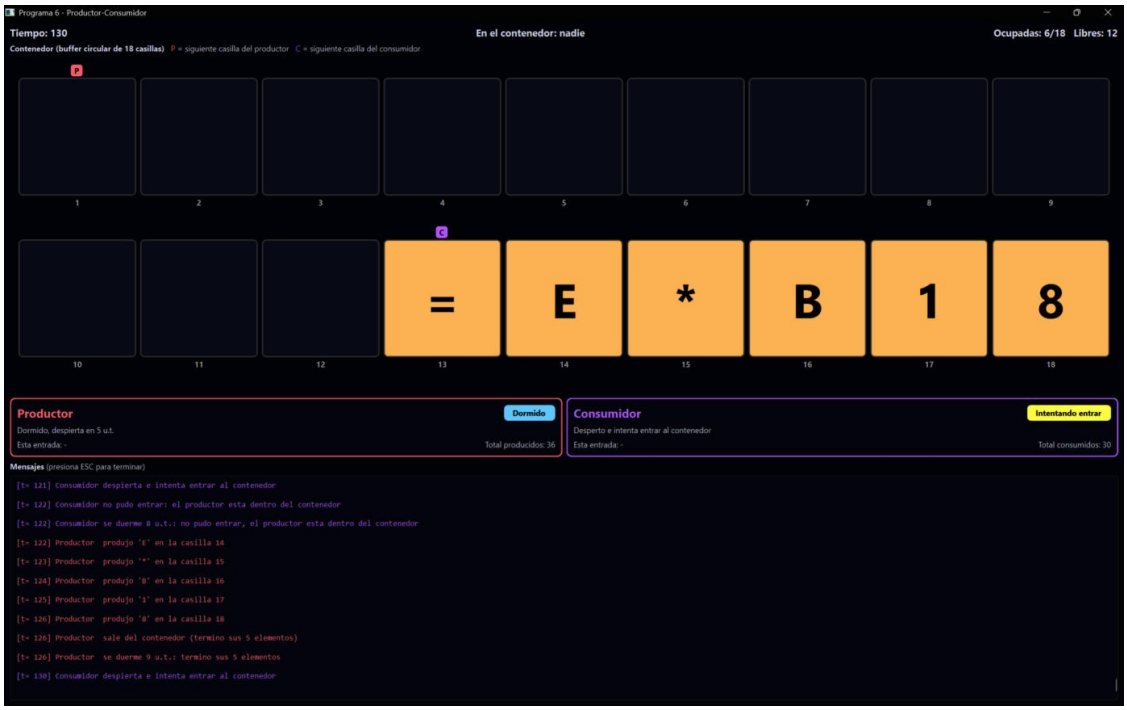


Ilustración 3. Intento fallido: el consumidor no puede entrar

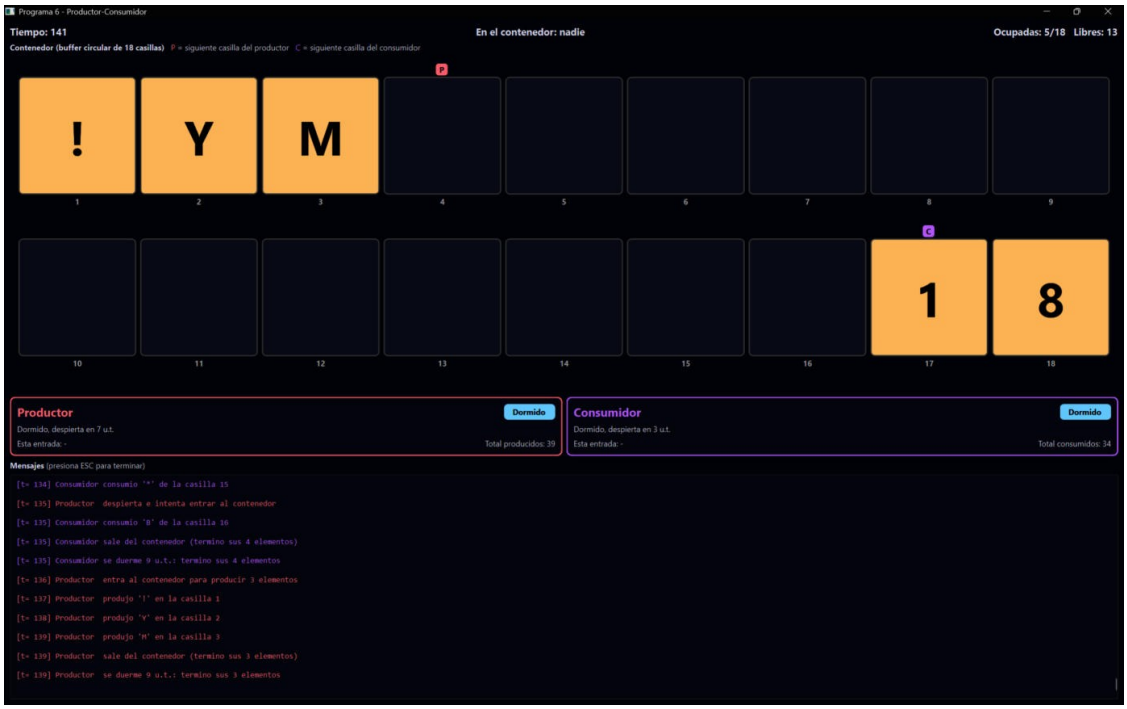


Ilustración 4. Buffer circular: el productor regresa a la casilla 1

Conclusiones

El problema Productor-Consumidor muestra por qué un recurso compartido necesita reglas de acceso: sin exclusión mutua, el productor y el consumidor podrían modificar el contenedor al mismo tiempo, y sin las condiciones de espacio y de producto se escribiría sobre casillas ocupadas o se leerían casillas vacías. En la simulación se observa cómo los tiempos aleatorios de sueño provocan intentos fallidos cuando el otro está dentro, y cómo el contenedor se llena cuando el productor es más rápido o se vacía cuando lo es el consumidor.

El buffer circular permite reutilizar las mismas 18 casillas indefinidamente con solo dos índices y la operación módulo, y garantiza que los elementos se consuman en el mismo orden en que se produjeron. Separar el modelo de la interfaz permitió probar estas propiedades sin abrir la ventana.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10a ed.). Wiley.

Stallings, W. (2018). Operating Systems: Internals and Design Principles (9a ed.). Pearson.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4a ed.). Pearson.

The Qt Company. (2024). PySide6 Reference Documentation — QTimer, QPainter y QListWidget. Recuperado de la documentación oficial de Qt for Python.